Student 17 **Mohammed Subahaan H** 192572165RC 2 / 2 submitted

SIMATS Engineering • CSE • CSA0802 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Submissions Report

Q1. Student Grade Book Application

CODE

```
students = {
    "Alice": [92, 88, 95, 90, 85],
    "Bob": [78, 65, 72, 80, 75],
    "Charlie": [55, 60, 45, 58, 62],
    "David": [88, 92, 79, 85, 91],
    "Emma": [40, 55, 48, 52, 45],
    "Frank": [70, 75, 68, 72, 74]
}
def get_grade(average):
    if average ≥ 90:
        return "O"
    elif average ≥ 80:
        return "A+"
    elif average ≥ 70:
        return "A"
    elif average ≥ 60:
        return "B+"
    elif average ≥ 50:
        return "B"
    else:
        return "F"
results = {
    name: {
        "marks": marks,
        "total": sum(marks),
        "average": round(sum(marks) / len(marks), 2),
        "grade": get_grade(sum(marks) / len(marks))
    }
    for name, marks in students.items()
}
topper = max(results.items(), key=lambda x: x[1]["average"])

num_subjects = len(next(iter(students.values())))
subject_names = [f"Subject {i+1}" for i in range(num_subjects)]

subject_means = [
    sum(marks[i] for marks in students.values()) / len(students)
    for i in range(num_subjects)
]

best_subject_index = subject_means.index(max(subject_means))
best_subject = subject_names[best_subject_index]

sorted_results = sorted(results.items(), key=lambda x: x[1]["average"], reverse=True)

print("-"*30)
print("STUDENT GRADE BOOK REPORT".center(30))
print("-"*30)

for name,data in sorted_results:
    marks_str=",".join(map(str,data["marks"]))
    print(f"Student Name : {name}")
    print(f"Marks        : {marks_str}")
    print(f"Total        : {data['total']}")
    print(f"Average      : {data['average']}")
    print(f"Grade        : {data['grade']}")
    print("-"*30)

print(f"Class Topper :  {topper[0]}({topper[1]['average']})")
print(f"Best Performing Subject :         {best_subject}(Mean: {round(max(subject_means),2)})")

print("\nSubject-wise Mean Marks:")
for subj,mean in zip(subject_names,subject_means):
    print(f"  {subj} : {round(mean,2)}")

print("="*30)
```

OUTPUT

```
(no output)
```

Q2. Debugging and Rewriting

CODE

```
"""Bug 1: Missing indentation for function bodies
Bug 2: Wrong operator used in deposit function(=+)
      the correct operator is (+=)
Bug 3: The condition balance > amount wrongly blocks withdrawing the exact balance amount;
        it should be balance ≥ amount to allow full withdrawal.
Bug 4: Incorrect slicing in mini_statement"""

"""DEBUGGED CODE"""
accounts = {}

def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}

def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))

def withdraw(acc_no, amount):
    if accounts[acc_no]['balance'] ≥ amount:
        accounts[acc_no]['balance'] -= amount
        accounts[acc_no]['txns'].append(('Withdraw', amount))
    else:
        print('Insufficient balance')

def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5:]:
        print(t)

create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```

OUTPUT

```
(no output)
```